MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión B del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_medina_miguel
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/medina_miguel/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
En la radiografía de un objeto, ¿qué representa un atributo y qué representa un método? Da un ejemplo de cada uno.

Tu respuesta

```
Atributo: Es lo que puede hacer el objeto.

Metodo: Es el objeto.
```

Corrección. Los conceptos están invertidos: lo que puede hacer el objeto es un método, no un atributo, y un método no es el objeto. Además faltan ejemplos. Un atributo guarda parte del estado (por ejemplo, la velocidad de un coche) y un método es un comportamiento (por ejemplo, acelerar).

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué hace el método main y por qué se declara static en los programas que usamos en clase?

Tu respuesta

```
el metodo main es lo que abre la funcion central del codigo que nos permite inicar el prgrama.
El static se utiliza indicar que la funcion que se creo esta establecida para ese parametro.
```

Corrección. Es correcto que main es el punto desde el que inicia el programa. La explicación de static es incorrecta: se declara static para que la JVM pueda invocarlo desde la clase sin crear antes una instancia.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un atributo de instancia y uno static? Explica qué comparten dos objetos de la misma clase.

Tu respuesta

```
Un atributo de instancia solo existe en una ejecucion de su linea y desaparece hasta ser nuevamente solicictado, mientras que uno static esta establecido para usarse desde que se crea hasta finaliar el programa.
```

Corrección. La descripción del atributo de instancia es incorrecta: no desaparece al terminar una línea, sino que vive mientras exista su objeto. Es válido que el static dura todo el programa, pero falta la idea central: cada objeto tiene su propia copia del atributo de instancia, y el static pertenece a la clase y todos los objetos lo comparten, como un contador de objetos creados.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo primitivo y qué guarda una de referencia? Da un ejemplo de cada una.

Tu respuesta

```
una variable de tipo primitiva son aquellas que se les asigna un valor inicial o toman un valor por defecto, mientras que los de que guradan por referencia guardan la direccion de memoria.
```

Corrección. Es válido que la variable de referencia guarda la dirección (la referencia) de un objeto. En la primitiva no se dice que guarda el valor mismo, y faltan los ejemplos: un int guarda directamente un 8 y una variable String guarda la referencia al objeto de texto.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace javac con un archivo .java y qué hace la JVM con el resultado?

Tu respuesta

```
javac ejecuta y crea el archivo .class de un archivo .java
```

Corrección. Es correcto que javac genera el .class a partir del .java, pero javac no ejecuta nada: compila el código fuente a bytecode. Falta la segunda parte: la JVM ejecuta ese bytecode.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[] datos = {7, 5, 3};
int[] alias = datos;
```

```
int i = 0;
while (i < 3) {
    System.out.println(alias[i] / 2);
    i++;
}
alias = new int[]{9, 8, 6};
System.out.println("B" + datos[0] + alias[0]);
double mitad = (double) (datos[1] / 2);
System.out.println(mitad);
```

Tu respuesta

```
1.- priemro imprime 3, segundo 2 y ahi se queda por que no hay un igual que siga avanando para que se divida por 3.
2.- imprime 16
3.- 2.0
```

Corrección. Las líneas 3, 2 y 2.0 son correctas. Falta la tercera línea: el while sí llega a i = 2 e imprime 3/2 = 1. En la cuarta línea se sumaron 7 y 9 para dar 16, cuando debía concatenarse "B" + 7 + 9 = B79; los valores 7 y 9 sí están bien identificados. La salida exacta era 3 / 2 / 1 / B79 / 2.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int modo = 1;
switch (modo) {
    case 1: System.out.println("LISTO");
    case 2: System.out.println("FIN");
}
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
No hay break que termine la sentencia de cada case, por tanto compila pero seguira entrando con las demas sentencias.
```

Corrección. La clasificación es incorrecta: el código funciona y no truena. La regla es correcta: sin break la ejecución sigue al siguiente case. Faltó la salida: LISTO / FIN.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
{ int nivel = 4; }
System.out.println(nivel);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
Las {} hacen que el int exista solo dentro de las mismas llaves haciendo que no compile por un error de sintaxys.
```

Corrección. Correcto: no compila, porque nivel solo existe dentro de las llaves donde se declara. Técnicamente es un error de ámbito (símbolo no encontrado) y no de sintaxis, pero la causa está bien identificada.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String etiqueta = null;
System.out.println(etiqueta.trim());
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
Imprmime null que es un string guardado en un string.1
```

Corrección. Incorrecto: compila pero truena. Llamar trim() sobre una referencia null lanza NullPointerException, y no se imprime nada (tampoco "null").

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada cargador fija su capacidad al fabricarse y nace vacío. Cuenta ranuras ocupadas y cargadores fabricados. Escribe colocar() y una consulta de ocupadas. Supón que solo se llama colocar cuando hay espacio. No uses arreglos.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué es final y qué static: capacidad o fabricados? ¿Alguno necesita ambos?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
cargador = new int Static;
```

Justificaciones

Sin respuesta.

> **Corrección.** La línea escrita no es una clase ni modela el cargador, y no hay justificaciones. Se esperaban: final int capacidad y int ocupadas de instancia, static int fabricados; un constructor con this.capacidad = capacidad, ocupadas = 0 y fabricados++; los métodos colocar() { ocupadas++; } y una consulta que devuelva ocupadas. En el reto, capacidad es final y fabricados es static; ninguno necesita ambos.

R4 · Código con repetición

Pregunta

El arreglo intentos contiene exactamente tres códigos enteros, en el orden en que se probaron. Devuelve el número del primer intento que coincide con clave (1, 2 o 3). Si ninguno coincide, devuelve 0. Usa do-while y deja de revisar al acertar. El arreglo no es null. No lo modifiques.

```
static int intentoValido(int[] intentos, int clave)
```

Ejemplos (argumentos → retorno):

{13, 42, 8}, 42 → 2 · {42, 8, 9}, 42 → 1 · {1, 2, 3}, 42 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
int CONTRASEÑA = 1234;

do{
    System.out.println("Ingrese la contraseña");
    clave = nextInt();

    if(clave ! contraseña)
        for(int i = 0; i <= 3; i++){
            System.out. println("te quedan: "+ i + " intentos")
        }
```

```
}while (clave == contraseña){
    return = "0";
}
```

Corrección. Falta la cabecera static int intentoValido(int[] intentos, int clave) y el arreglo no se recorre: lee del teclado e imprime, cosa que el enunciado prohíbe. Se reconocen el uso de do-while y la idea de comparar un intento con la clave. La condición while (clave == contraseña) está invertida y no se detiene al acertar; tampoco hay numeración desde 1 ni return 0 al fracasar. La idea esperada era un índice i dentro de un do-while, que en cada vuelta revise si intentos[i] == clave para hacer return i + 1, avance mientras i < 3 y al final haga return 0.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Devuelve la acción del módulo con switch sobre Modo. Primero: si bateria < 10, devuelve "CARGAR", incluso con modo null. En otro caso, null devuelve "SIN MODO" antes del switch. El enum ya existe: escribe solo el método. No imprimas ni leas del teclado.

```
enum Modo { APAGADO, MEDIR, CALIBRAR, PRUEBA }
static String accion(Modo modo, int bateria)
```

Regla o valor recibido	Texto devuelto
APAGADO	"DETENER"
MEDIR	"LEER"
CALIBRAR	"AJUSTAR"
PRUEBA	"SIMULAR"

Tu respuesta

```
System.out.println("Que acion desea realizar");
Modo modo.nextline();
System.out.println("Cuanta bateria tienes");
bateria.nextInt();

if(bateria < 10){
    System.out.println("CARGAR BATERIA");
}else{
    switch(modo){
        case 1:
            System.out.printl("DETENER");
        break;
        case 2:
            System.out.println("LEER");
        break;
        case 3:
            Sytem.out.println("AJUSTAR");
        break;
        case 4:
            System.out.println("Simular");
```

```
        break;
        default

        break;

    }
}
```

Corrección. Bien: revisa bateria < 10 antes del switch, lo que da prioridad incluso con null, y usa break en cada case. Las cuatro acciones están en el orden correcto, pero los case usan 1-4 en lugar de APAGADO, MEDIR, CALIBRAR y PRUEBA, y además escribe "Simular" en vez de "SIMULAR". También falla en esto: no hay cabecera, lee del teclado e imprime en vez de devolver con return, escribe "CARGAR BATERIA" en vez de "CARGAR" y no devuelve "SIN MODO" cuando modo es null y la batería alcanza. En la sintaxis: printl, Sytem y default sin dos puntos.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.